Group 18 Project Report: Pet Recognition

Bousmar Cyril
63401800
cyril.bousmar@uclouvain.be

Kouff Samuel
41162300
samuel.kouff@student.uclouvain.be

Robert Séverin
19902301
severin.robert@student.uclouvain.be

I. Introduction

In the context of the course LELEC2885, we were tasked to implement[1] a deep learning method to generate segmentation masks out of The Oxford-IIIT Pet Dataset [1]. This is covered by Part A. Part B answers an open research question about the uncertainty estimation of the model from part A. This report is structured as followed. Section II describes chosen networks. Section III explains conducted workflow to select, train and test models. Section IV presents obtained results and highlights model's strengths and flaws. Section V presents the uncertainty study of the best performing model from part A. Section VI summarizes and discusses our findings, and suggests possible future work.

II. Networks - Part A

[2] covers a broad range of models architectures, specifically in regard to segmentation. Most promising models are: U-Net [3], PSPNet [4], Mask R-CNN [5], and Deeplabv3+ [6]. We selected them for two reasons.

Firstly, they are all representatives of their categories, being state-of-the-art implementation with the highest performances and accuracies, as of now, or close in the past.

Secondly, they have different architectures, making them interesting to compare, and useful material to learn from.

A. U-Net

This model, in figure 6, consists of a contracting path to capture context and a symmetric expanding path that enables precise localization. It consists of the repeated application of two 3x3 convolutions (unpadded convolutions), each followed by a rectified linear unit (ReLU) and a 2x2 max pooling operation with stride 2 for downsampling. At each downsampling step we double the number of feature channels. Every step in the expansive path consists of an upsampling of the feature map followed by a 2x2 convolution ("up-convolution") that halves the number of feature channels, a concatenation with the correspondingly cropped feature map from the contracting path, and two 3x3 convolutions, each followed by a ReLU. The final layer is a 1x1 convolution that output the wished number of classes.

B. Pyramid Scene Parsing Network (PSPNet)

This model, in figure 7, uses a pyramid pooling module to exploit the capability of global context information by different-region based context aggregation. The pyramid pooling module is a four-level one with bin sizes of 1×1, 2×2, 3×3 and 6×6 respectively. The feature map obtained to feed the pyramid pooling module is created by a *ResNet50* network (`PyTorch` pre-trained model).

C. Others

Both *Mask-RCNN* and *DeepLabV3+* are under implementation, but not completed by the time of this report. Therefore, they are neither described, nor compared and appearing in results. However, they are available on the GitHub.

III. Training - Part A

The based network provided with the canvas of the project (CNN) is used as benchmark. All models are trained on an `NVIDIA GTX1070 Ti 8Go`.

A. Loss function

Binary Cross Entropy Loss is used for CNN and U-Net. It measures pixel-wise differences between the predicted probabilities and the actual binary labels in an image. Each pixel is predicted to either belong to the foreground (pet), 1, or not, 0 (background).

$$BCE = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log(p_i) + (1-y_i)\cdot\log(1-p_i)\right]$$

PSPNet uses `BCEWithLogitsLoss` which takes logits as input, non-normalized scores before application of activation function (like sigmoid).

B. Optimizer

Selected optimizer is Adam. A reference and very effective for large datasets and complex models. Its learning rate is a hyperparameter. It is set to 0.001 by default, as in [3] and [4].

C. Hyperparameters

Other than the learning rate from the optimizer, hyperparameters are: image size, batch size, number of epochs, number of in- and out-channels of the model, and the number of classes.

In this case, the number of classes is fixed to 1, as there is only 1 pet per image, thus removing the need of multiple mask classes.

The number of in- and out-channels concerns only PSPNet. These are set following [4], which matches this dataset case.

[1] Link to the project: https://github.com/c-bousmar/pet_recognition

As explained in [3], section 3, to optimize GPU memory usage and minimize overhead, large input tiles are prioritized. Here, images of size 224x224 are used over the largest possible batch size, 8. This being conditioned by the GPU. The same parameters are used for all models.

Models are trained over a subsequently large number of epochs. 50 for CNN and 20 for U-Net and PSPNet.

D. Over-fitting

Figures 8, 9 and 10 clearly show signs of over-fitting in the training process. To mitigate its effect, models weight are saved at each epoch, which allows in depth analysis of loss function values over training and validation test to select which epoch was the best, before presenting signs of over-fitting. The weight of this epoch is then selected.

E. Cross-validation

To improve model's performance while preventing over-fitting, models are trained in multiple folds. In this case, and because of how resource intensive the training is, this process was done manually for this report. All values of each fold and each epoch were saved and compared. For future use convenience, automation has been added afterwards.

IV. Results - Part A

Each image probability matrices receive a threshold transformation to obtain a binary matrix which represents a mask applied to the image to distinguish between background and subject.

A. Qualitative Evaluation

Three representative images are chosen, and their predictions from each model are compared. Figure 1 shows an ideal case, and figures 2 and 3 show respectively prediction failures for U-Net and PSPNet.

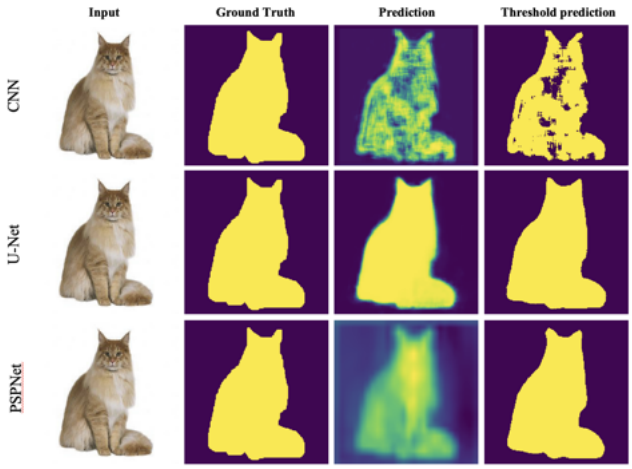

Fig. 1: Predictions of each models for `img_13.png`. This case is ideal. The background and the pet are very distinct, and all models perform well.

When subject and background are very distinct, it models perform very well.

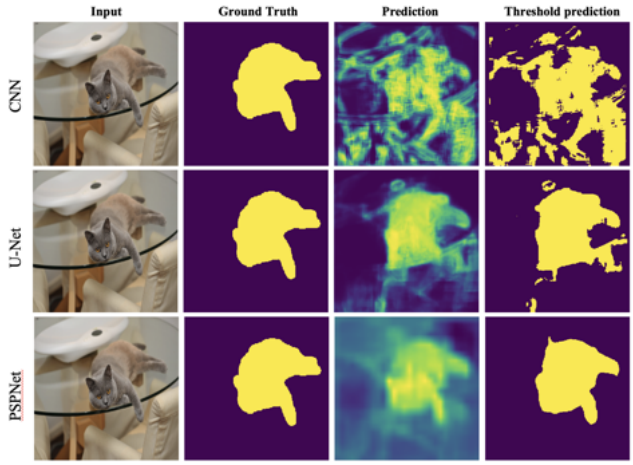

Fig. 2: Predictions of each models for `img_28.png`. U-Net struggles to differentiate background and subject. But, PSPNet proves to be much better at finding the subject in the image.

But in more complex situations, with complex background, reflexions, or similar background and subject, it is not the case. U-Net struggles with complex backgrounds and PSPNet struggles with high similitude of background and subject.

This is probably due to their output nature, where U-Net uses very extreme values for each pixel (only close to 0 or 1), whereas PSPNet uses a much wider range (and so much more pixels are at an intermediate value). Therefore, when the first model could nearly not need a threshold to create a mask, it is essential to have a good one for the second. It seems that, depending on the situation, the threshold value of PSPNet should change to achieve best results overall.

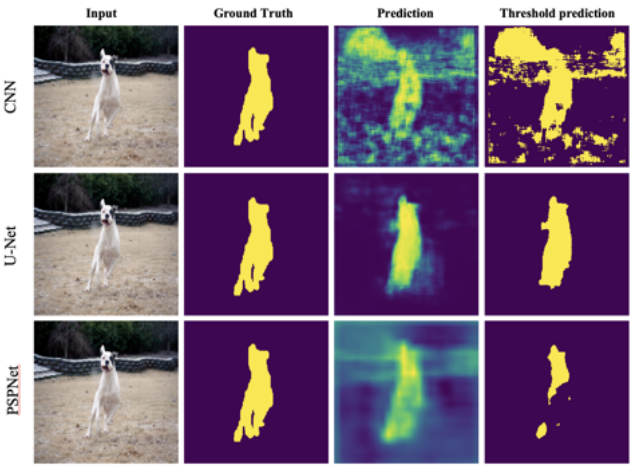

Fig. 3: Predictions of each models for `img_29.png`. PSPNet finds the subject, but is not able to differentiate all pixels, especially those in the color range of the background. But here, U-Net performs well.

Finally, it is interesting to note that pet's hair (i.e. cat mustache) are very hard to detect for models. But in this case, it seems that the ground truth is at fault, as it considers very wide areas for which only very thin and small parts are from the subject.

B. Quantitative Evaluation

Empirical comparison between models is based on the two following metrics. Dice coefficient and Intersection over Union (IoU):

$$Dice = \frac{2|P \cap GT|}{|P| + |GT|} = \frac{2TP}{2TP + FP + FN},$$

$$IoU = \frac{|P \cap GT|}{|P \cup GT|} = \frac{TP}{TP + FP + FN}$$

These two coefficients takes into account the same parameters, but not with the same distribution. Dice puts more emphasis on true positives, while IoU penalize more false positives and negatives. But they fluctuate in the same way, just not at the same rate.

Models are also compared based on their size, the needed number of epochs for training and the Training Time Per Epoch (TTPE). This allows to have a better understanding for their usage.

Model	mDice	mIoU	Epoch	TTPE	Size
CNN	0.76 ±0.13	0.63 ±0.15	47	**1:07**	**0.217**
UNet	0.86 ±0.11	0.77 ±0.14	**11**	3:15	121
PSPNet	**0.91 ±0.1**	**0.86 ±0.12**	13	4:23	196

TABLE I: Empirical comparison between reference model (CNN) and the two implemented models (U-Net and PSPNet). Best results are in bold. Means and deviations of both Dice and IoU are computed based on the value obtained for each image on the test set.

PSPNet performs much better overall, but U-Net is not that far off. Table I shows that U-Net is trained faster and is smaller, which places it in a good position whenever PSPNet is too heavy. In any case, basic CNN is not that good, except when it comes to size.

V. Uncertainty estimation - Part B

With the best model, PSPNet, transformations are applied at testing time on the dataset and compared to their predictions to obtain an estimate of its uncertainty.

A. Transformations

Nine different transformations are implemented to test certainty and robustness of PSPNet. Four transformations change the position of the subject in the image: horizontal and vertical flips, and left and right 90-degree rotations. The others change the pixel information: brightening, darkening, adding noise, greyscale, and image blur.

B. Entropy

To calculate the pixel entropy, the cross entropy is computed for each of the 10 predictions (the original image + 9 transformations). The formula is as follows, with P the probability of a pixel to be part of the subject:

$$entropy = -P\log(P + 1e-8) - (1-P)\log(1 - P + 1e-8)$$

By getting the mean of each pixel's entropy, an image with the certainty of each pixel can be generated. This shows how much each of the predictions agree on the classification of the pixel.

C. Results

Figure 4 shows that the model is more impacted by the transformations that tweak the pixel information like brighten, darken and noise. On the other hand, rotations have an impact on the probability of trees to be part of the dog. There is a clear decrease of probability for the vertical flip and the -90° rotation.

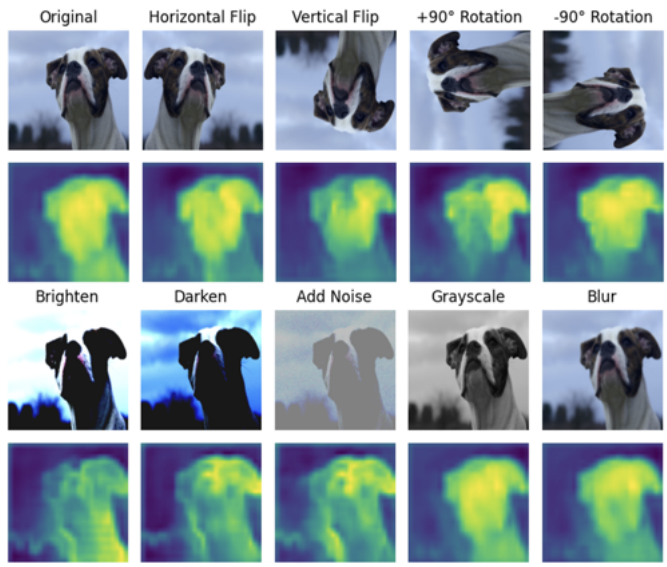

Fig. 4: Transformations and their respective prediction

Figure 5 shows entropy results on images. Lighter areas indicate greater uncertainty in pixel prediction.

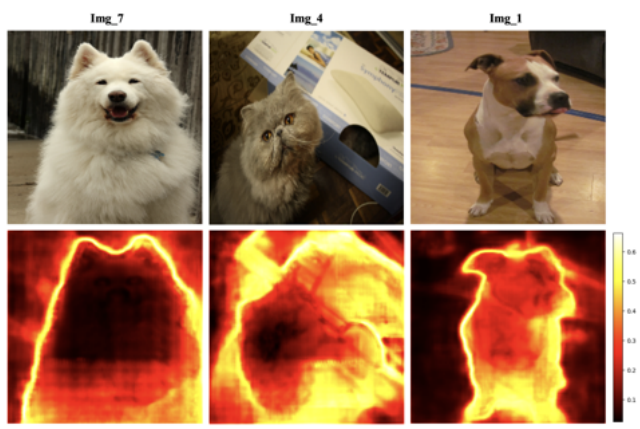

Fig. 5

Three parts of the images can lead to an increase in the model uncertainty.

First, as expected, the contour of the subject (in *Img_7*). Second, the shadow of the subject (in *Img_1*). Third, objects in the image (in *Img_4*), which have major impact. *Img_4*

is the worst-case scenario, combining the three components, inducing a lot of uncertainty in the predictions of the model.

VI. Discussion and conclusions

Out of all tested models, PSPNet performs the best (19.79% better than CNN and 5.81% better than U-Net). It also promises even better results in a case where the threshold can be automatically fine-tuned based on the situation at hand.

Some situations are trickier for the models: complex background, subject shadow, very high/low luminosity, reflections, and very similar back- and foreground. Introducing transformation for data augmentation at training time could improve the results, since some transformations at testing time improved the predictions (vertical flip and left 90º the most). Finally, as no change was noticed between original and gray scaled image, working with the later could improve the speed of models, both at training and evaluation.

Future works could involve the fine-tuning of hyper-parameters for the best model, PSPNet, or the widening of the range of studied models to *Mask-RCNN* [5], *DeepLabV3+* [6] (implementation for both is already undergoing), or *Global Context Vision Transformer* [7].

Adahessian [8] or *Yogi* [9] optimizers could be used to improve training and potentially accuracy.

Finally, as only self trained models have been studied here, their comparison with fine-tuned pre-trained models like *Mask-RCNN* from `PyTorch` library would be interesting.

References

[1] O. M. Parkhi, A. Vedaldi, A. Zisserman, C. V. Jawahar. Cats and dogs. In *IEEE conference on computer vision and pattern recognition*, 2012, pp 3498–3505. IEEE.

[2] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz and D. Terzopoulos. Image Segmentation Using Deep Learning: A Survey. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022, vol. 44, no. 7, pp. 3523-3542. IEEE.

[3] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, 2015, pp. 234–241. Springer.

[4] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia. Pyramid scene parsing network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2881–2890. IEEE.

[5] K. He, G. Gkioxari, P. Dollár and R. Girshick. Mask R-CNN. In *IEEE International Conference on Computer Vision*, 2017, pp. 2980-2988. ICCV.

[6] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European conference on computer vision*, 2018, pp. 801–818. ECCV, 2023.

[7] A. Hatamizadeh, H. Yin, G. Heinrich, J. Kautz, P. Molchanov. Global context vision transformers. In *Proceedings of the International Conference on Machine Learning*, 2023, pp. 12633–12646. PMLR.

[8] Z. Yao, A. Gholami, S. Shen, M. Mustafa, M., Keutzer, K., Mahoney, M. (2021). ADAHESSIAN: An Adaptive Second Order Optimizer for Machine Learning. Proceedings of the AAAI Conference on Artificial Intelligence, 35(12), 10665-10673.

[9] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar. Adaptive Methods for Nonconvex Optimization. In *Advances in Neural Information Processing Systems*, 2018, vol. 31. Curran Associates, Inc.

Appendix

A. Networks architectures

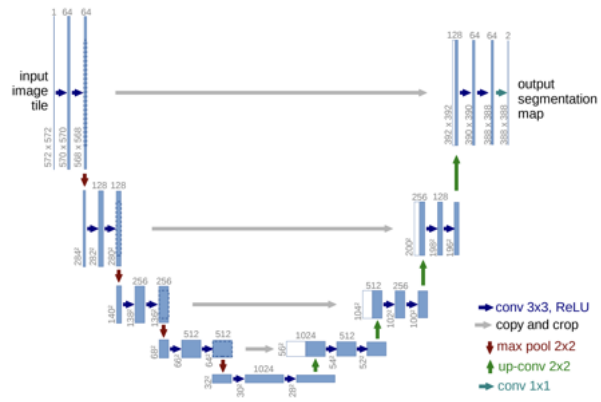

Fig. 6: U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

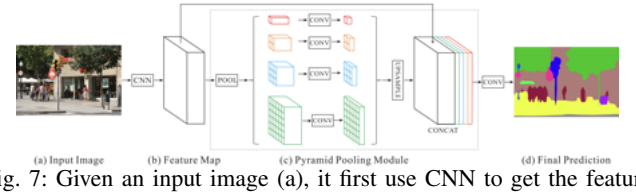

Fig. 7: Given an input image (a), it first use CNN to get the feature map of the last convolutional layer (b), then a pyramid parsing module is applied to harvest different sub-region representations, followed by upsampling and concatenation layers to form the final feature representation, which carries both local and global context information in (c). Finally, the representation is fed into a convolution layer to get the final per-pixel prediction (d).

B. Train and validation curves for over-fitting analysis of models

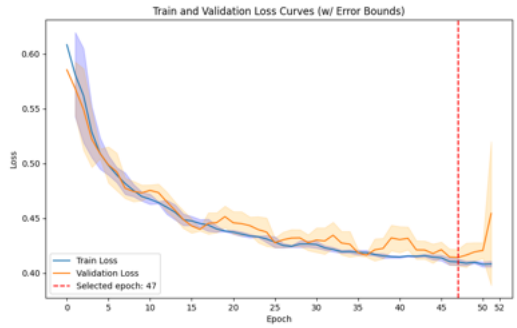

Fig. 8: Learning curves of basic CNN (benchmark model). At each epoch, the model is trained and then evaluated. Curves represent the mean values with a standard deviation over multiple folds of cross-validation process. Selected weights saved for the models are at the epoch represented by a red line.

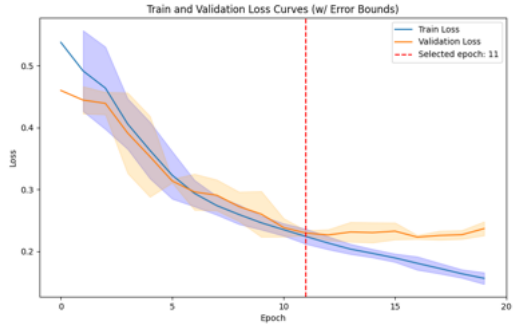

Fig. 9: Learning curves of U-Net. At each epoch, the model is trained and then evaluated. Curves represent the mean values with a standard deviation over multiple folds of cross-validation process. Selected weights saved for the models are at the epoch represented by a red line.

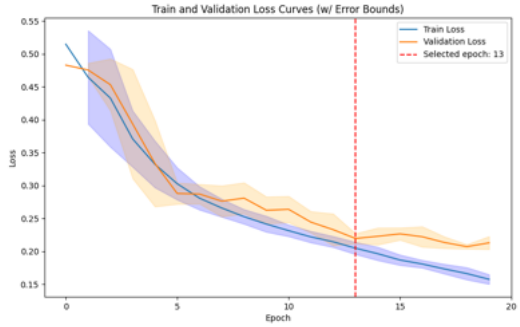

Fig. 10: Learning curves of PSPNet. At each epoch, the model is trained and then evaluated. Curves represent the mean values with a standard deviation over multiple folds of cross-validation process. Selected weights saved for the models are at the epoch represented by a red line.